Aircraft Family Recognition Using Deep Learning and Grad-CAM

Deep Learning Semester Project Report

Muhammad Ishtiaq Haider

October 2026

Live web app	https://aircraft-family-gradcam.vercel.app/
GitHub repository	https://github.com/Muhammad-Ishtiaq-Haider/aircraft-family-gradcam
Dataset	FGVC-Aircraft (Oxford VGG), family-level labels, 70 classes
Final model	EfficientNet-B0 (ImageNet-pretrained, fine-tuned), 4.10 M parameters
Headline result	79.96% top-1 and 94.06% top-5 accuracy on the official test set (3,333 images)

Abstract

This project builds an image classifier that names the aircraft family shown in a photograph, a fine-grained recognition task in which many classes differ only in small details such as engine placement, tail shape or window layout. Using the public FGVC-Aircraft benchmark with its official train, validation and test split, three ImageNet-pretrained convolutional networks (MobileNetV2, ResNet18 and EfficientNet-B0) were fine-tuned and compared. The model was selected on validation accuracy only and evaluated once on the untouched test set. EfficientNet-B0 performed best, reaching 79.96% top-1 accuracy, 94.06% top-5 accuracy and 78.21% macro F1. Predictions are explained with Grad-CAM heatmaps, and a deletion test on 300 images checks that the highlighted regions matter to the model: hiding the most-highlighted quarter of the pixels lowered the true-class probability by 0.738 on average, against 0.007 for the least-highlighted quarter. The trained model is exported to ONNX and runs entirely in the browser in a public web application, and all code, weights and results are available in an open GitHub repository.

1. Introduction

1.1 Problem statement

Given a single photograph of an aircraft, the task is to predict its family, for example Boeing 737, Airbus A320, Cessna 172 or Eurofighter Typhoon. The problem is a 70-way image classification task. It is fine-grained because the classes share the same overall structure (fuselage, wings, tail, engines) and are told apart by subtle visual cues, and because photographs vary widely in viewpoint, distance, lighting, airline livery and background.

1.2 Motivation

Recognising aircraft types automatically is useful as a building block for spotting and cataloguing tools, educational applications and the organisation of large photo collections. Beyond the application, the task is a good test of transfer learning on a small, fine-grained dataset, and it raises a question that matters for any deployed model: does the network look at the aircraft, or at irrelevant context such as the sky, the airport or a logo? This project therefore pairs accuracy with an explicit explainability analysis.

1.3 Objectives

- Fine-tune and fairly compare three pretrained CNN architectures on the same data, split and training recipe.
- Follow a clean evaluation protocol: choose the model with the validation set and report the test set once.
- Explain predictions with Grad-CAM and test whether the explanations are faithful to the model.
- Analyse errors, deploy the model as a public browser demo and publish all code, weights and results for reproduction.

2. Background

Transfer learning. Networks pretrained on ImageNet (Deng et al., 2009) learn general visual features such as edges, textures and object parts. Fine-tuning all layers on a small target dataset usually works much better than training from scratch, which is important here because there are only about 48 training images per family on average.

Architectures. ResNet18 (He et al., 2016) uses residual connections and has 11.2 M parameters. MobileNetV2 (Sandler et al., 2018) uses inverted residual blocks and depthwise convolutions to stay small (2.3 M parameters). EfficientNet-B0 (Tan and Le, 2019) was found by architecture search and scales depth, width and resolution in a balanced way (4.1 M parameters in this 70-class configuration).

Grad-CAM. Gradient-weighted Class Activation Mapping (Selvaraju et al., 2017) explains a prediction by combining the feature maps A^k of a late convolutional layer. For a class c with score y^c, the weight of each map is the spatial average of the gradient, a_k^c = mean over (i, j) of the derivative of y^c with respect to A^k_{ij}, and the heatmap is L^c = ReLU(sum over k of a_k^c A^k). The map is upsampled to the image size and normalised to the range 0 to 1. Its resolution equals that of the chosen layer, which is 7 x 7 for a 224 x 224 input in all three networks.

3. Dataset

The project uses FGVC-Aircraft (release 2013b) by Maji, Rahtu, Kannala, Blaschko and Vedaldi of the Oxford Visual Geometry Group (arXiv:1306.5151), a public benchmark and therefore a secondary dataset. It has a three-level label hierarchy (manufacturer, family and variant); this project uses the family level with 70 classes, which gives a task that is challenging but still learnable from limited data. The dataset is downloaded automatically by torchvision (`FGVCAircraft`) and is not stored in the repository because it is about 2.6 GB.

Property	Value
Source	https://www.robots.ox.ac.uk/~vgg/data/fgvc-aircraft/
Label level / classes	Family / 70
Split (official, unchanged)	3,334 train / 3,333 validation / 3,333 test (10,000 images)
Training images per family	about 48 on average; class sizes vary (see class_distribution.png in the repository)
Content	civil airliners, business jets, propeller aircraft and military aircraft
Licence	non-commercial research and education only

Copyright banner. Every image carries a 20-pixel copyright banner along the bottom edge. The banner is cropped away before any processing, so the model cannot use that text as a shortcut.

Official split. Using the benchmark's own split keeps the results comparable with other work and avoids leakage between training and evaluation images.

4. Methodology

4.1 Preprocessing and augmentation

After the banner is removed, evaluation images are resized to 224 x 224 pixels (stretching, without cropping, so the whole aircraft stays visible) and normalised with the ImageNet mean and standard deviation. Training images use a random resized crop to 224 x 224 with scale between 0.5 and 1.0, a random horizontal flip and light colour jitter (brightness, contrast and saturation 0.2).

4.2 Models

Model	Parameters (M)	Classifier head replaced	Grad-CAM layer
MobileNetV2	2.31	Linear(1280, 70)	features[-1]
ResNet18	11.21	Linear(512, 70)	layer4[-1]
EfficientNet-B0	4.10	Linear(1280, 70)	features[-1]

All three start from ImageNet weights and every layer is fine-tuned.

4.3 Training setup

Setting	Value
Optimiser	AdamW, maximum learning rate 5e-4, weight decay 1e-2
Schedule	OneCycleLR (15% warm-up)
Loss	Cross-entropy with label smoothing 0.1
Epochs / batch size	20 / 64 (mixed-precision training)
Seed	42 (same for all models)
Hardware	Google Colab, Tesla T4 GPU; about 26 minutes per model
Software	PyTorch 2.11, torchvision 0.26, Python 3.13

4.4 Evaluation protocol

The checkpoint of each model with the highest validation accuracy is kept, and the final model is the one with the highest validation accuracy (ties would go to the smaller model). The test set plays no part in any decision and is evaluated once, after selection. Metrics are top-1 accuracy, top-5 accuracy and macro F1 (the unweighted mean of per-class F1, which gives rare families the same weight as common ones). A normal-approximation 95% confidence interval for accuracy on n = 3,333 test images is about plus or minus 1.4 percentage points.

4.5 Explainability

Grad-CAM is computed on the last convolutional block of each network using the gradient of the predicted class score. To test whether the heatmaps are faithful, a **deletion test** is run on 300 correctly classified test images: the 25% of pixels with the highest Grad-CAM values are set to the dataset mean colour and the true-class probability is re-measured, and the same is done for the 25% of pixels with the lowest values. If the heatmaps are meaningful, hiding the most important region should reduce the probability far more than hiding the least important one.

4.6 Export and deployment

The selected model is exported to ONNX (opset 17) and verified against PyTorch on 100 test images: all 100 predictions agree and the largest absolute difference in any logit is 4.1e-05. The web application loads the ONNX file with ONNX Runtime Web and applies the same resize and normalisation in JavaScript, so predictions are computed inside the visitor's browser and uploaded photos are never sent to a server.

5. Results

5.1 Model comparison

Model	Val acc	Test top-1	Test top-5	Macro F1	Params (M)	Train time
MobileNetV2	77.17%	78.04%	93.52%	76.42%	2.31	26.3 min
ResNet18	76.36%	77.02%	91.84%	75.16%	11.21	25.5 min
EfficientNet-B0 (selected)	**78.64%**	**79.96%**	**94.06%**	**78.21%**	**4.10**	**26.1 min**

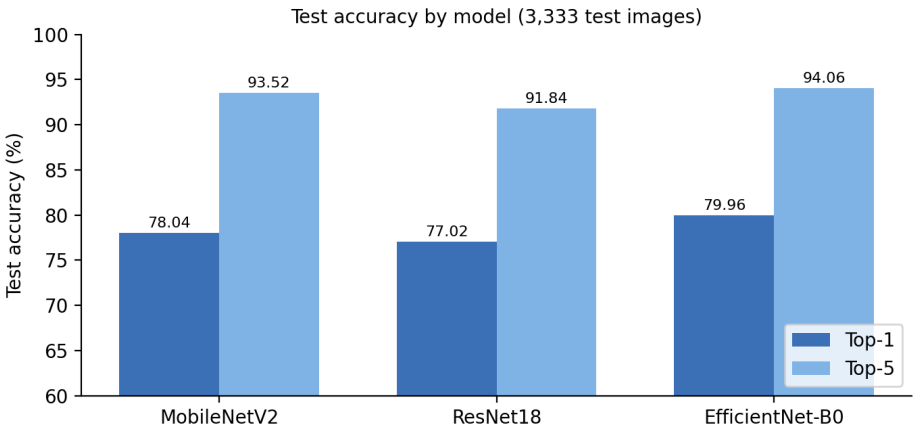


Figure 1. Test top-1 and top-5 accuracy of the three models (values from results.json).

EfficientNet-B0 achieved the best score on every metric while using fewer than half the parameters of ResNet18. Its accuracy of 79.96% has an approximate 95% interval of 78.6% to 81.3%. The lead over ResNet18 (2.9 points) is larger than that sampling interval, whereas the lead over MobileNetV2 (1.9 points) is smaller and should be read as suggestive rather than conclusive, because each model was trained once with one seed. Validation accuracy (78.64%) and test accuracy (79.96%) are close, so selecting on the validation set did not overfit to it. Top-5 accuracy of 94.06% means the correct family is almost always among the five most likely answers.

5.2 Training dynamics

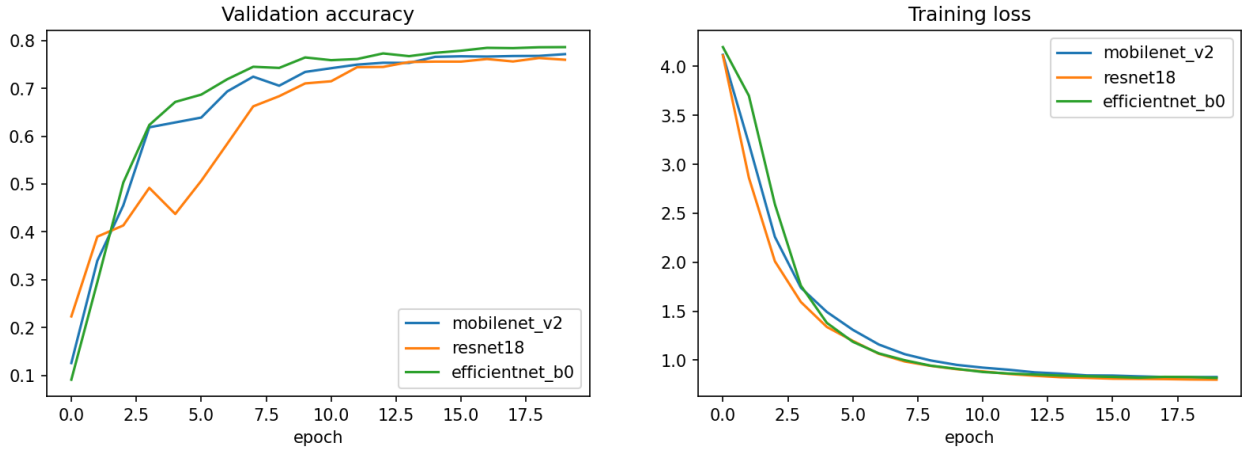


Figure 2. Validation accuracy (left) and training loss (right) over 20 epochs.

All three models trained smoothly. Validation accuracy rises quickly in the first ten epochs and flattens by about epoch 15, so longer training would add little. EfficientNet-B0 leads from about epoch 3 onwards. ResNet18 starts highest but dips around epoch 4 and recovers most slowly. The training loss ends near 0.8; with label smoothing of 0.1 over 70 classes the lowest possible cross-entropy is about 0.74, so the models fit the training set closely. The remaining test errors therefore look more like a generalisation limit (few images per family, visually similar classes) than a sign of too little training; this is an interpretation of the curves rather than a separately measured result.

6. Error analysis

The most frequent confusions of the final model on the test set are listed below.

True family	Predicted as	Count
C-47	DC-3	16
Boeing 777	Boeing 767	11
A320	A330	9
MD-90	MD-80	7
MD-80	MD-90	7
MD-11	DC-10	7
Boeing 707	Boeing 747	7
A320	Boeing 737	7
Fokker 50	Dash 8	6
Embraer E-Jet	Boeing 737	6

The errors are sensible. C-47 versus DC-3 is largely a labelling ambiguity, because the C-47 is the military transport version of the DC-3 and the two look almost identical. The 777 and 767, the MD-80 and MD-90, and the MD-11 and DC-10 are related designs from the same manufacturer with similar fuselage and tail layouts. A320 confused with A330 or Boeing 737 shows that even different manufacturers' narrow-body and wide-body airliners are hard to separate from some viewpoints.

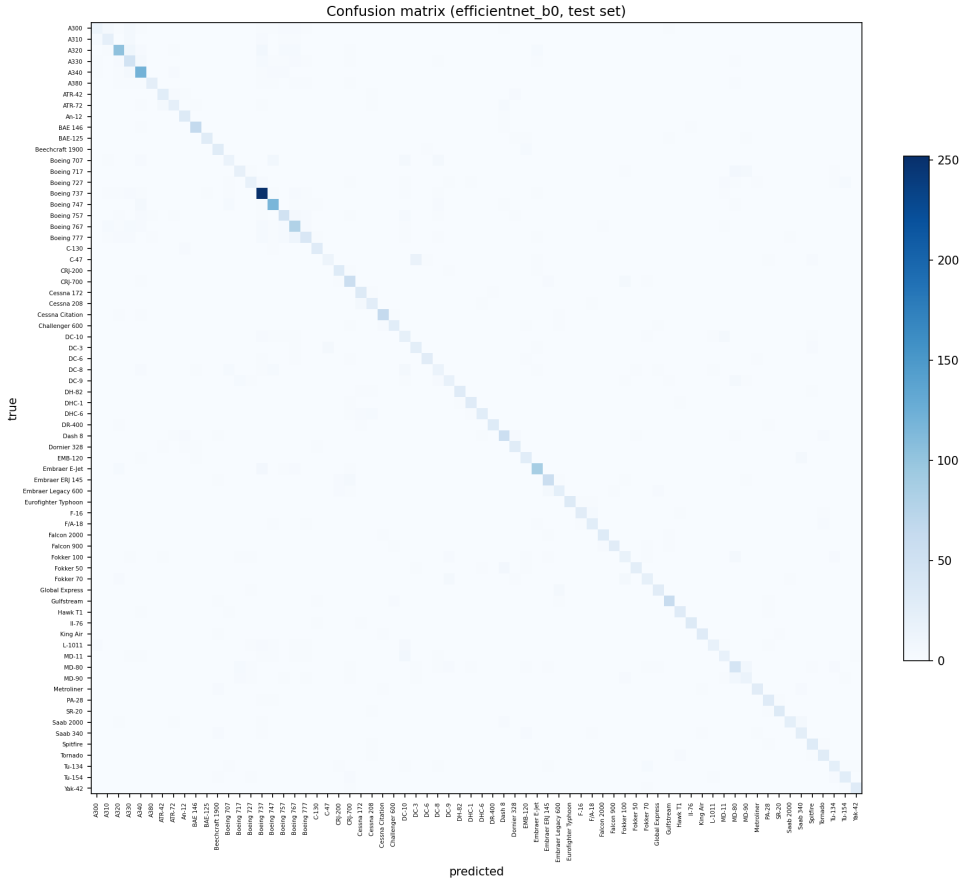


Figure 3. Confusion matrix on the test set (EfficientNet-B0). The strong diagonal shows most families are recognised correctly; errors are scattered with a few visible clusters (Boeing 737 row, A300/A310/A320, MD-80/MD-90). The darkest cell is Boeing 737 simply because it has the most test images, so raw counts favour frequent classes.

7. Grad-CAM analysis

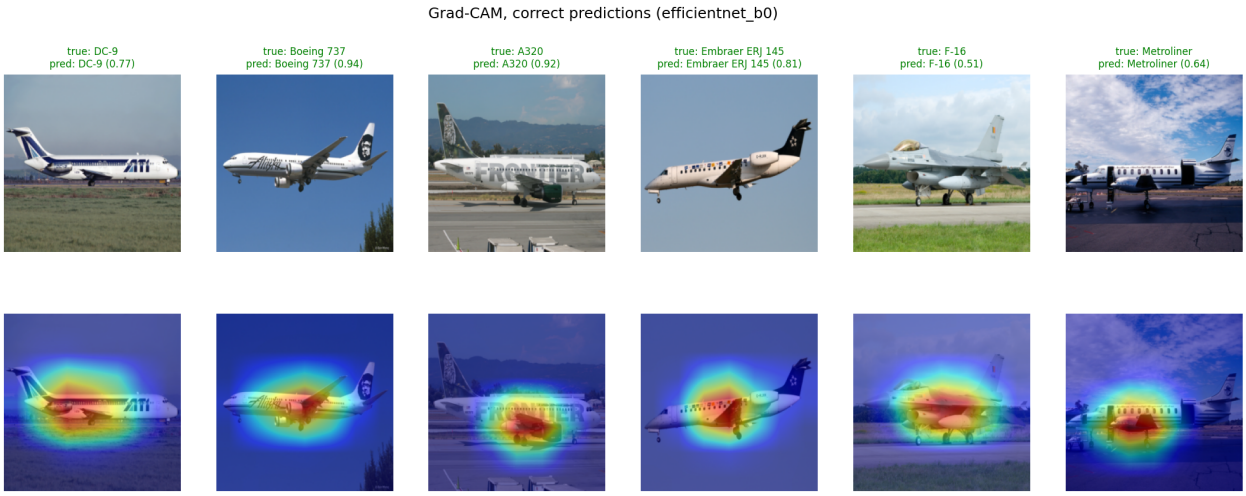


Figure 4. Grad-CAM for six correctly classified test images (top row: photo with true and predicted family and confidence; bottom row: heatmap overlay).

In all six correct examples the strongest activation lies on the aircraft body, mostly the central fuselage around the wings and engines, while sky, grass and tarmac stay cold. The heat blobs are broad and near the image centre. This reflects the 7 x 7 resolution of the last feature map and probably some centre bias, since aircraft are usually centred in this dataset. Noses and tails, which people often use to tell types apart, receive only light coverage, so the maps should not be read as proof that the model uses a specific part.

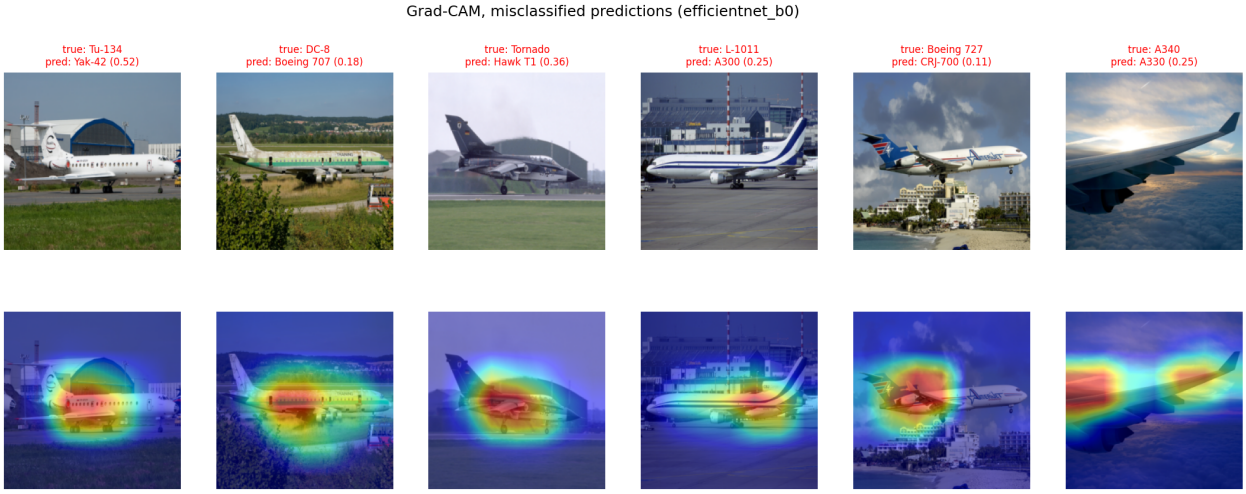


Figure 5. Grad-CAM for six misclassified test images (true family, predicted family and confidence above each photo).

In most of the six misclassified examples the heat is still on the aircraft, so the errors are not simply caused by looking at the background. In some, the heat spreads onto context, such as the hangar roof in the Tu-134 example and vegetation in the DC-8 example. One image shows only a wing seen from a window (A340 predicted as A330), a very hard view. Several wrong answers are reasonable look-alikes (Tu-134 as Yak-42, DC-8 as Boeing 707). The confidence on these errors is low (0.11 to 0.52) compared with the correct examples (0.51 to 0.94). This is based on twelve images and is an observation, not a general statistic.

Deletion test. On 300 correctly classified test images, hiding the most important 25% of pixels (according to Grad-CAM) lowered the true-class probability from 0.771 to roughly 0.033, a mean drop of 0.738. Hiding the least important 25% lowered it by only 0.007. The most-important region hurt

more than the least-important one in 300 of 300 images.

Condition	Mean change in true-class probability
Baseline (no masking), mean true-class probability	0.771
Hide the 25% most important pixels	drop of 0.738
Hide the 25% least important pixels	drop of 0.007
Images where the important region hurts more	300 / 300 (100%)

This supports the claim that the heatmaps point at evidence the model genuinely uses. It is a sanity check and not a proof: masking also changes image statistics, the large drop partly reflects that a quarter of the picture is replaced by a flat colour, and Grad-CAM is a coarse method. The baseline probability of 0.771 is not close to 1 because label smoothing deliberately keeps predicted confidences lower.

8. Deployment and reproducibility

Web application. The site at https://aircraft-family-gradcam.vercel.app/ is a static page hosted on Vercel. It presents the dataset and code details, the results and Grad-CAM gallery, links to the report, presentation, video and repository, and a live demo. In the demo, a visitor chooses one of 12 test images or uploads a photo, and the ONNX model (about 16 MB) returns the five most likely families with probabilities. Everything runs client-side.

Repository. The public GitHub repository contains the notebook with its saved outputs, the PyTorch weights (.pth), the ONNX model, the class list and preprocessing settings (labels.json), result files and plots, sample images and Grad-CAM examples, two command-line scripts (`predict.py` for ONNX inference and `gradcam.py` for heatmaps) and a README with setup instructions.

Reproduction. Opening the notebook in Google Colab with a T4 GPU and running all cells downloads the dataset, trains the three models with the stated seed and settings, evaluates once on the test set and regenerates every file. Results may differ slightly between runs because GPU operations are not fully deterministic.

9. Limitations and future work

- **Single run.** Each model was trained once with one seed, so differences of about one point between models are within noise. Several seeds with mean and spread would be more convincing.
- **Benchmark-only evaluation.** All reported numbers come from the FGVC-Aircraft test split. The model was not systematically tested on outside photographs, which may differ in camera, angle and lighting, and accuracy there may be lower.
- **No rejection option.** The classifier always answers with one of its 70 families, so a photo that is not an aircraft, or shows an unseen type, still receives a family label.
- **Resolution and aspect ratio.** Images are stretched to 224 x 224, which distorts wide aircraft photos. Higher input resolution and aspect-preserving resizing would probably help fine details.
- **Label granularity.** Family-level labels merge several variants, and near-identical designs (777 versus 767, MD-80 versus MD-90, C-47 versus DC-3) remain the main error source.
- **Explanation quality.** Grad-CAM at 7 x 7 resolution is coarse and centre-biased; finer methods or occlusion-based analysis could give sharper evidence.

Future work includes larger backbones and ensembles, multi-seed evaluation, a small collected test set of outside photographs, an out-of-scope detector, variant-level classification and a quantitative

comparison of explanation methods.

10. Conclusion

A fine-tuned EfficientNet-B0 recognises 70 aircraft families with 79.96% top-1 and 94.06% top-5 accuracy on the official FGVC-Aircraft test set, outperforming fine-tuned MobileNetV2 and ResNet18 under an identical recipe. The evaluation followed a strict protocol (selection on validation, a single test evaluation), the remaining errors are mostly between closely related designs, and Grad-CAM together with a deletion test indicates that the model relies on the aircraft itself rather than the background. The model, code and results are public, and a browser demo lets anyone try it. The main caveats are the single training run and the lack of a systematic test on photos from outside the benchmark.

References

[1] S. Maji, E. Rahtu, J. Kannala, M. Blaschko, A. Vedaldi. Fine-Grained Visual Classification of Aircraft. arXiv:1306.5151, 2013.

[2] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, D. Batra. Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. ICCV, 2017.

[3] M. Tan, Q. V. Le. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML, 2019.

[4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.-C. Chen. MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR, 2018.

[5] K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. CVPR, 2016.

[6] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei. ImageNet: A Large-Scale Hierarchical Image Database. CVPR, 2009.

[7] I. Loshchilov, F. Hutter. Decoupled Weight Decay Regularization. ICLR, 2019.

[8] L. N. Smith, N. Topin. Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates. 2019.

[9] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna. Rethinking the Inception Architecture for Computer Vision. CVPR, 2016.

Appendix: sources of the reported numbers

All figures in this report come from the project's results.json file and the plots in the repository (results folder). Figure 1 was drawn from the values in the results table; Figures 2 to 5 are the plots produced by the notebook. The report text was drafted with AI assistance (Claude) and checked against those result files.